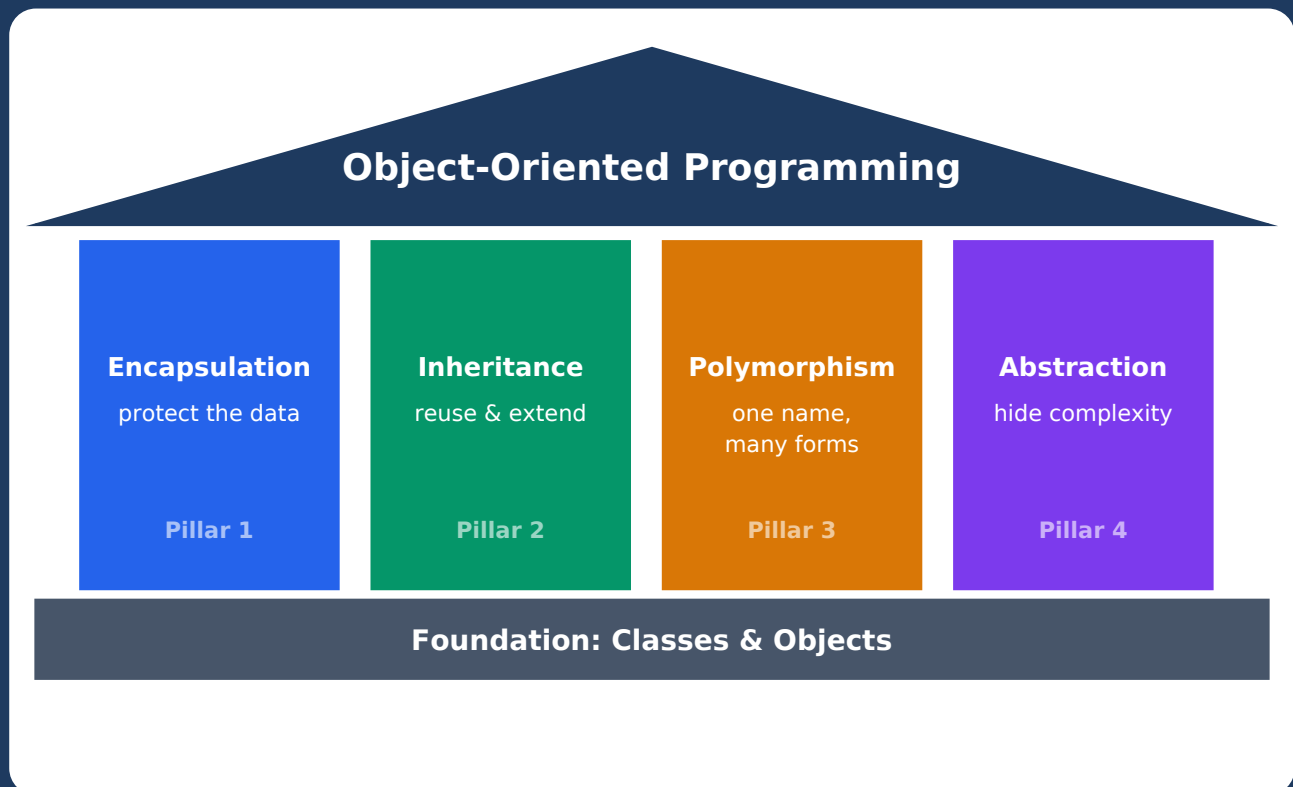


A COMPLETE BEGINNER'S GUIDE

Object-Oriented Programming in C++

Why OOP is needed, and the four pillars explained step by step



- 17 complete C++ programs, each compiled and run, with real output
- Diagrams, real-life analogies and comparison tables
- A mini project that uses all four pillars together
- Interview questions, practice exercises and a one-page cheat sheet

Contents

1. Why Do We Need OOP?	5
1.1 What is a programming paradigm?	5
1.2 The problem with procedural code	5
1.3 The OOP idea: bundle data with behaviour	6
1.4 What OOP gives you	7
1.5 Where OOP is used	7
1.6 An honest note	7
2. Classes and Objects: The Foundation	8
2.1 Class vs object	8
2.2 Your first class	9
2.3 Constructors and destructors	10
2.4 The this pointer	11
2.5 struct vs class in C++	12
2.6 Common beginner mistakes	12
3. Encapsulation	14
3.1 Access specifiers	14
3.2 Example: a safe bank account	15
3.3 Getters and setters	16
3.4 Why encapsulation matters	17
3.5 Encapsulation vs data hiding	17
4. Inheritance	18
4.1 Syntax	18
4.2 Example: animals	18
4.3 Modes of inheritance	20
4.4 Types of inheritance	20
4.5 Constructor and destructor order	21
4.6 Multiple inheritance	22
4.7 Inheritance vs composition (is-a vs has-a)	23
4.8 What is not inherited	24
5. Polymorphism	25
5.1 Two kinds of polymorphism	25
5.2 Compile-time: function overloading	25
5.3 Compile-time: operator overloading	26

5.4 Run-time: the problem without virtual	27
5.5 Run-time: virtual functions to the rescue	28
5.6 How does C++ do this? (vtable, simply)	29
5.7 Overloading vs overriding	29
6. Abstraction	31
6.1 Abstract classes and pure virtual functions	31
6.2 Interfaces in C++	33
6.3 Abstract class vs interface	35
6.4 Abstraction vs encapsulation	35
7. Putting It All Together: A Library System	36
8. The Four Pillars Side by Side	39
8.1 How they work together	39
9. Interview Questions and Answers	41
10. Practice Exercises	43
11. Cheat Sheet and Glossary	45
11.1 Syntax at a glance	45
11.2 Glossary	46

How to use this guide

This guide assumes you know basic C++: variables, `if/else`, loops and functions. You do **not** need to know anything about classes yet. Read the chapters in order, because each one builds on the last.

- **Chapter 1** explains the problem OOP solves. Do not skip it: if you understand *why*, the four pillars will feel natural instead of like rules to memorise.
- **Chapter 2** teaches classes and objects, the foundation everything else stands on.
- **Chapters 3 to 6** cover the four pillars: Encapsulation, Inheritance, Polymorphism and Abstraction. Each has a definition, an everyday analogy, complete code with its real output, common mistakes and a recap.
- **Chapter 7** builds a small library system that uses all four pillars at once.
- **Chapters 8 to 11** are for revision: a side-by-side comparison, interview questions, practice exercises and a cheat sheet.

Tip

Type the programs yourself instead of copying them. Then change something and predict the output before you run it. That habit teaches more than reading.

To compile any example: `g++ -std=c++17 file.cpp -o app` then run `./app` (on Windows: `app.exe`). Online compilers such as onlinegdb.com also work.

How to read the code boxes

Grey boxes are C++ source code. The dark box right after a program is its **actual output**: every program in this guide was compiled with g++ 13 and run, and the output was copied in automatically, so what you see is exactly what you should get.

Each pillar has its own colour throughout the guide:

Pillar	Colour	One-line idea
Encapsulation	Blue	Keep data private; change it only through safe methods.
Inheritance	Green	Build a new class on top of an existing one (is-a).
Polymorphism	Amber	One interface, many behaviours.
Abstraction	Purple	Show what something does, hide how it does it.

1. Why Do We Need OOP?

Before learning *how* object-oriented programming works, it is worth understanding *why* it was invented. OOP is a solution to a real problem, and that problem only becomes visible once programs get big.

1.1 What is a programming paradigm?

A **paradigm** is a style or way of organising a program. The same task can be written in different styles, just as the same story can be told as a novel, a comic or a film. The two styles that matter here are:

- **Procedural programming**: the program is a list of steps and functions that operate on data stored elsewhere. C is the classic procedural language.
- **Object-oriented programming (OOP)**: the program is a collection of **objects**. Each object bundles its own data together with the functions that work on that data. C++, Java, C# and Python all support OOP.

C++ supports both styles, which is why it is a great language for seeing the difference clearly.

1.2 The problem with procedural code

Imagine writing banking software in a purely procedural way. Account balances are ordinary variables, and functions like `withdraw()` take a balance and return a new one:

C++ · procedural.cpp

```
#include <iostream>
using namespace std;

// Procedural style: data lives in loose variables,
// functions live somewhere else. Nothing ties them together.
double balanceRiya = 5000;
double balanceAarav = 12000;

double withdraw(double balance, double amount) {
    return balance - amount;          // no checks at all!
}

int main() {
    balanceRiya = withdraw(balanceRiya, 9000);    // takes more than she has
    balanceAarav = -999999;                       // any line can do this

    cout << "Riya: " << balanceRiya << endl;
    cout << "Aarav: " << balanceAarav << endl;
    return 0;
}
```

OUTPUT

```
Riya: -4000
Aarav: -999999
```

Two serious bugs happened, and the compiler did not complain about either:

1. Riya withdrew more money than she had, so her balance became negative. The `withdraw()` function had no way to enforce the rule, and even if it did, nothing forces other code to call it.
2. Aarav's balance was overwritten directly. In a real project with 200 files and 20 programmers, any one of them could write a line like that, and finding it would take days.

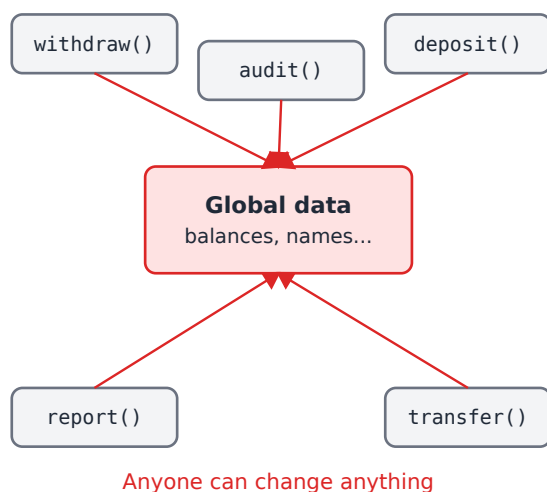
In small programs this is manageable. In large programs, procedural code tends to develop these problems:

Problem	What it looks like in practice
Unprotected data	Global or shared variables can be changed from anywhere, so one careless line breaks the whole program.
Data and logic are scattered	The balance lives in one place, the rules about balances live in five other files. To understand one concept you must read everything.
Copy-paste code	A savings account and a current account are 90% the same, so the code gets duplicated. Fix a bug in one copy, forget the other.
Hard to extend	Adding a new account type means editing many existing functions full of <code>if (type == ...)</code> checks, and each edit risks breaking something that worked.
Does not match how we think	We think in things (a customer, an account, a loan), not in loose variables and unrelated functions.

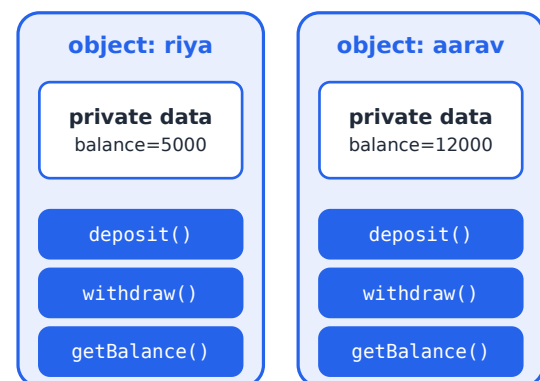
1.3 The OOP idea: bundle data with behaviour

OOP's core idea is simple: **put the data and the functions that work on that data together, in one unit called an object**, and let the object protect its own data. Instead of a loose `balance` variable, you have an account object that owns its balance and refuses invalid withdrawals.

Procedural: shared data, loose functions



Object-oriented: data + functions together



Data can only change through the object's methods

Figure 1.1 Procedural code shares data with every function. OOP gives each object its own data and a small set of methods that guard it.

We will rebuild this exact bank example the OOP way in Chapter 3 and see both bugs become impossible.

1.4 What OOP gives you

Benefit	What it means	Bank example
Data security	Objects hide their data and only allow safe changes.	Balance cannot go negative because only <code>withdraw()</code> can change it, and it checks.
Modularity	Each class is a self-contained unit you can understand, test and fix on its own.	A bug in interest calculation is in the <code>Account</code> class, nowhere else.
Reusability	Write common code once and reuse it through inheritance and composition.	<code>SavingsAccount</code> and <code>CurrentAccount</code> both reuse <code>Account</code> .
Easy to extend	Add new types without rewriting old, tested code.	Add <code>FixedDeposit</code> next year without touching existing classes.
Maintainability	Changes stay local, so large codebases stay manageable.	Changing how balance is stored affects one class only.
Real-world modelling	Code is organised like the problem domain.	<code>Customer</code> , <code>Account</code> , <code>Transaction</code> map to real things.
Teamwork	Different people can build different classes that talk through clear interfaces.	One developer builds <code>Loan</code> , another builds <code>Account</code> .

1.5 Where OOP is used

Almost every large piece of software you use is built with OOP ideas: game engines (Unreal Engine is written in C++, where every enemy, weapon and player is an object), web browsers like Chrome and Firefox, desktop applications, banking and booking systems, operating system drivers, and mobile apps.

1.6 An honest note

OOP is a tool, not a religion. A 30-line script that renames files does not need classes. OOP pays off when a program is large, lives for years, has several people working on it, or models many related kinds of things. Those are exactly the programs you will write in industry, which is why OOP is taught everywhere and asked about in almost every interview.

Key takeaways

- A paradigm is a style of organising code. OOP organises code around objects.
- Procedural code keeps data and functions apart, which makes large programs fragile.
- OOP bundles data with the functions that use it, and lets the object protect its own data.
- The main payoffs: security, modularity, reuse, easy extension and maintainability.

2. Classes and Objects: The Foundation

Every one of the four pillars is built from two ideas: the **class** and the **object**. Understand these clearly and the rest of the guide will be straightforward.

2.1 Class vs object

A **class** is a blueprint or template. It describes what data something will hold and what it can do, but it is not a thing by itself. An **object** is a real thing built from that blueprint, with its own actual values. One class can produce any number of objects.

Real-life analogy

An architect draws **one house plan** (the class). The builder then builds **many houses** from it (objects). Every house has the same rooms, but each has its own paint colour, owner and address. Painting one house red does not paint the others.

Likewise, a cookie cutter is the class and each cookie is an object.

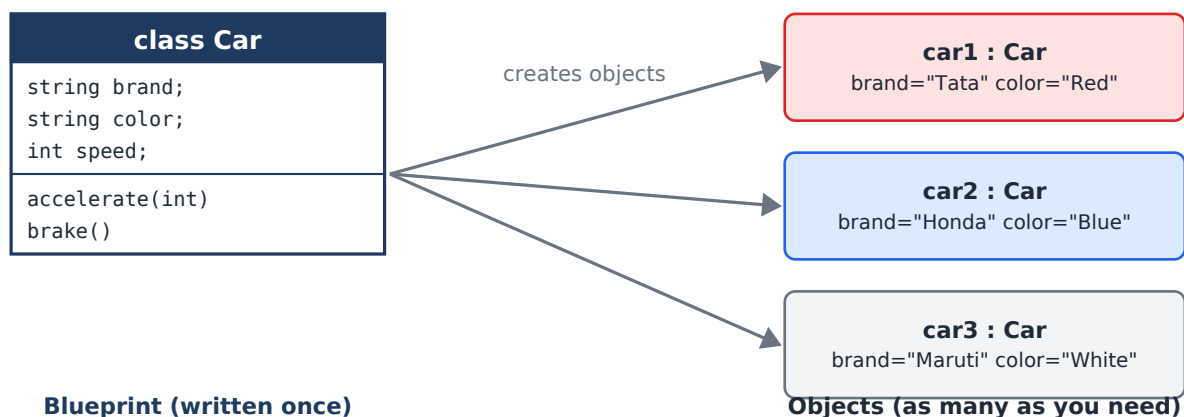


Figure 2.1 One class, many objects. Each object has its own copy of the data.

Term	Meaning	In the Car example
Class	Blueprint that defines data and behaviour	<code>class Car { ... };</code>
Object (instance)	A real variable created from a class	<code>Car car1("Tata", "Red");</code>
Attribute (data member)	A variable that belongs to the object	<code>brand, color, speed</code>
Method (member function)	A function that belongs to the object	<code>accelerate(), brake()</code>
Constructor	Special method that sets up a new object	<code>Car(string b, string c)</code>
Dot operator .	Accesses a member of an object	<code>car1.accelerate(40);</code>

Term	Meaning	In the Car example
State	The current values of an object's attributes	car1 is red, going 40 km/h
Behaviour	What the object can do	speed up, stop

2.2 Your first class

C++ · car.cpp

```
#include <iostream>
#include <string>
using namespace std;

class Car {
public:
    // Attributes (data / state)
    string brand;
    string color;
    int speed;

    // Constructor: runs automatically when an object is created
    Car(string b, string c) {
        brand = b;
        color = c;
        speed = 0;
    }

    // Methods (behaviour)
    void accelerate(int amount) {
        speed = speed + amount;
        cout << brand << " is now going " << speed << " km/h" << endl;
    }

    void brake() {
        speed = 0;
        cout << brand << " has stopped." << endl;
    }
};

int main() {
    Car car1("Tata", "Red");    // object 1
    Car car2("Honda", "Blue");  // object 2

    car1.accelerate(40);
    car2.accelerate(60);
    car1.accelerate(20);
    car1.brake();

    cout << car2.brand << " is " << car2.color
        << " and still at " << car2.speed << " km/h" << endl;
    return 0;
}
```

OUTPUT

```
Tata is now going 40 km/h
Honda is now going 60 km/h
Tata is now going 60 km/h
Tata has stopped.
Honda is Blue and still at 60 km/h
```

Line-by-line explanation

- `class Car { ... };` declares a new type called `Car`. Note the **semicolon after the closing brace**: forgetting it is the most common beginner error.
- `public:` means everything below it can be used from outside the class. (Chapter 3 explains why we usually make data `private` instead.)
- `string brand;` `string color;` `int speed;` are the attributes. Every `Car` object gets its own copy of each.
- `Car(string b, string c)` is the **constructor**. It has the same name as the class and no return type, and it runs automatically when you write `Car car1("Tata", "Red");`.
- `accelerate()` and `brake()` are methods. Inside them, `speed` means *this particular object's* speed.
- Look at the output: `car1` reached 60 and then stopped, but `car2` is still at 60. Each object keeps its **own state**.

2.3 Constructors and destructors

A **constructor** prepares a new object so it starts in a valid state. A **destructor** (name starts with `~`) runs automatically when the object is destroyed, and is used to clean up (close files, free memory). C++ lets you write several constructors for different situations:

C++ · constructors.cpp

```
#include <iostream>
#include <string>
using namespace std;

class Student {
public:
    string name;
    int rollNo;

    // 1. Default constructor (no arguments)
    Student() {
        name = "Unknown";
        rollNo = 0;
        cout << "Default constructor called" << endl;
    }

    // 2. Parameterized constructor, using an initializer list
    Student(string n, int r) : name(n), rollNo(r) {
        cout << "Parameterized constructor called for " << name << endl;
    }

    // 3. Copy constructor (makes a new object from an existing one)
    Student(const Student& other) {
        name = other.name + " (copy)";
        rollNo = other.rollNo;
        cout << "Copy constructor called" << endl;
    }

    // Destructor: runs automatically when the object is destroyed
    ~Student() {
        cout << "Destructor called for " << name << endl;
    }

    void show() {
```

C++ · constructors.cpp

```
        cout << "Roll " << rollNo << ": " << name << endl;
    }
};

int main() {
    Student s1;           // default
    Student s2("Priya", 21); // parameterized
    Student s3 = s2;       // copy

    s1.show();
    s2.show();
    s3.show();
    cout << "--- end of main ---" << endl;
    return 0;             // destructors run here, in reverse order
}
```

OUTPUT

```
Default constructor called
Parameterized constructor called for Priya
Copy constructor called
Roll 0: Unknown
Roll 21: Priya
Roll 21: Priya (copy)
--- end of main ---
Destructor called for Priya (copy)
Destructor called for Priya
Destructor called for Unknown
```

- **Default constructor:** takes no arguments. Used by `Student s1;`.
- **Parameterized constructor:** takes arguments. The `: name(n), rollNo(r)` part is an **initializer list**, the preferred way to set members in C++.
- **Copy constructor:** builds a new object from an existing one, as in `Student s3 = s2;`.
- **Destructor:** notice the destructors run in the **reverse order** of creation (s3, then s2, then s1) when `main` ends.

Note

If you write no constructor at all, C++ provides a default one for you. But as soon as you write any constructor yourself, the automatic default constructor disappears, so `Student s1;` would no longer compile unless you also write `Student()` yourself.

2.4 The `this` pointer

Inside a method, `this` is a pointer to the object the method was called on. You need it when a parameter has the same name as a member, and it also lets a method return the object itself so calls can be chained.

C++ · this.cpp

```
#include <iostream>
#include <string>
using namespace std;

class Point {
private:
    int x, y;
public:
```

C++ · this.cpp

```
// Parameter names are the same as member names.
// 'this' is a pointer to the current object, so this->x
// means "the x that belongs to this object".
Point(int x, int y) {
    this->x = x;
    this->y = y;
}

// Returning *this lets us chain calls together
Point& moveBy(int dx, int dy) {
    x += dx;
    y += dy;
    return *this;
}

void print() const {
    cout << "(" << x << ", " << y << ")" << endl;
}

};

int main() {
    Point p(2, 3);
    p.print();
    p.moveBy(1, 1).moveBy(10, 0).print();    // chained calls
    return 0;
}
```

OUTPUT

```
(2, 3)
(13, 4)
```

2.5 struct vs class in C++

In C++ a **struct** can have methods and constructors too. The **only** difference is the default access level: members of a **struct** are **public** by default, members of a **class** are **private** by default. By convention, use **struct** for simple bundles of data and **class** for anything with rules to protect.

2.6 Common beginner mistakes

Mistake	Fix
Forgetting <code>;</code> after <code>class X { ... }</code>	Always end a class definition with <code>;</code> .
Writing <code>Car c();</code> to create an object	That declares a <i>function</i> . Write <code>Car c;</code> for the default constructor.
Giving a constructor a return type (<code>void Car()</code>)	Constructors have no return type at all.
Thinking changing one object changes the others	Each object has its own copy of the data members.
Using <code>.</code> with a pointer (<code>p.speed</code>)	Use <code>-></code> with pointers: <code>p->speed</code> , or <code>(*p).speed</code> .

Key takeaways

- A class is a blueprint; an object is a real instance with its own data.
- Attributes store state; methods define behaviour.
- Constructors set up an object; destructors clean it up (in reverse order).
- `this` points to the current object.

3. Encapsulation

Encapsulation means bundling data and the methods that operate on that data into a single unit (a class), and **restricting direct access** to that data from outside. The outside world can only interact with the data through a controlled set of public methods.

Real-life analogy

A medicine capsule wraps the medicine inside a shell: you swallow the capsule, you never handle the powder directly.

An ATM holds cash, but you cannot reach in and grab notes. You use the screen and keypad, and the machine checks your PIN and your balance before giving you anything. The cash is the private data; the keypad and screen are the public methods.

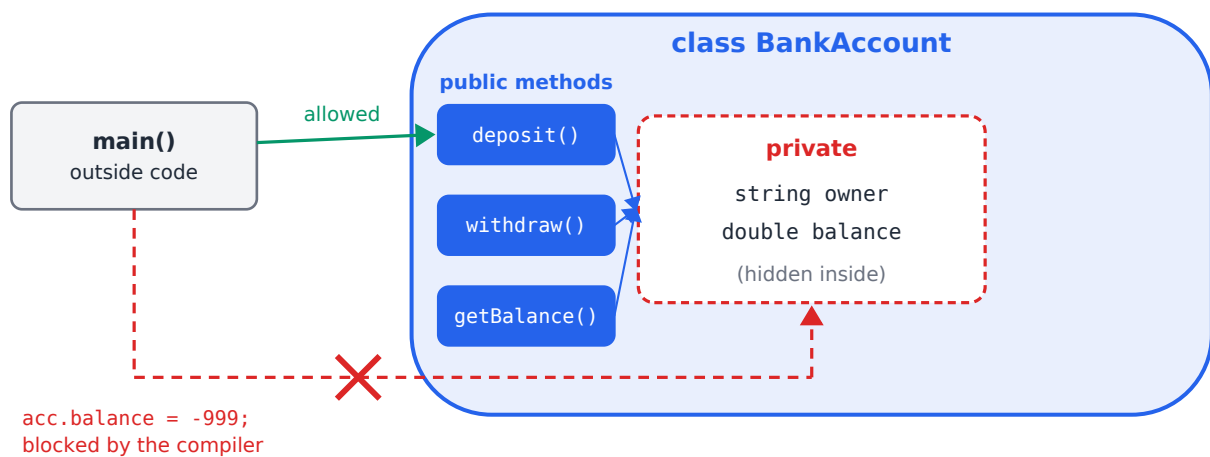


Figure 3.1 Outside code can call public methods but cannot touch private data directly.

3.1 Access specifiers

C++ enforces encapsulation with three **access specifiers**. They decide who is allowed to use each member:

Specifier	Same class	Derived (child) class	Outside code	Typical use
<code>private</code>	Yes	No	No	Data members (the default inside a <code>class</code>)
<code>protected</code>	Yes	Yes	No	Data children need but outsiders must not touch
<code>public</code>	Yes	Yes	Yes	The methods that form the class's interface

3.2 Example: a safe bank account

Here is the bank from Chapter 1, rebuilt with encapsulation. The balance is **private**, so the only way to change it is through **deposit()** and **withdraw()**, and those methods check every request.

C++ · bank.cpp

```
#include <iostream>
#include <string>
using namespace std;

class BankAccount {
private:                                // hidden: only this class can touch these
    string owner;
    double balance;

public:                                  // the safe, public "front door"
    BankAccount(string name, double openingBalance) {
        owner = name;
        balance = (openingBalance < 0) ? 0 : openingBalance;
    }

    // Getters: read-only access
    string getOwner() const { return owner; }
    double getBalance() const { return balance; }

    // Controlled ways to change the data
    void deposit(double amount) {
        if (amount <= 0) {
            cout << " [rejected] Deposit must be positive." << endl;
            return;
        }
        balance += amount;
        cout << " Deposited " << amount << endl;
    }

    bool withdraw(double amount) {
        if (amount <= 0 || amount > balance) {
            cout << " [rejected] Cannot withdraw " << amount << endl;
            return false;
        }
        balance -= amount;
        cout << " Withdrew " << amount << endl;
        return true;
    }
};

int main() {
    BankAccount acc("Riya", 5000);

    acc.deposit(2000);
    acc.withdraw(9000);    // more than the balance -> rejected
    acc.deposit(-50);      // nonsense amount -> rejected
    acc.withdraw(1500);

    // acc.balance = -999999;    // COMPILE ERROR: 'balance' is private

    cout << acc.getOwner() << "'s balance: " << acc.getBalance() << endl;
    return 0;
}
```

OUTPUT

```
Deposited 2000
[rejected] Cannot withdraw 9000
[rejected] Deposit must be positive.
Withdrew 1500
Riya's balance: 5500
```

Compare this with the procedural version. Both bugs from Chapter 1 are now impossible:

- **Overdrawing is rejected** because `withdraw()` checks the balance first, and it is the *only* path to change the balance.
- **Direct overwriting does not compile.** If you uncomment `acc.balance = -999999`; the compiler stops with: `error: 'double BankAccount::balance' is private within this context`. The bug is caught before the program even runs.

3.3 Getters and setters

A **getter** returns the value of a private member. A **setter** changes it, but only after checking that the new value makes sense. This is how a class guarantees its data is always valid (programmers call this keeping the class's **invariant**).

C++ · setter.cpp

```
#include <iostream>
#include <string>
using namespace std;

class Student {
private:
    string name;
    int marks;           // must always stay between 0 and 100

public:
    Student(string n) : name(n), marks(0) {}

    int getMarks() const { return marks; }

    // A setter is a guarded door: it checks before it changes
    void setMarks(int m) {
        if (m < 0 || m > 100) {
            cout << "Invalid marks " << m << " for " << name
                << " (ignored)" << endl;
            return;
        }
        marks = m;
    }
};

int main() {
    Student s("Kabir");
    s.setMarks(87);
    s.setMarks(150);    // rejected
    s.setMarks(-5);     // rejected
    cout << "Final marks: " << s.getMarks() << endl;
    return 0;
}
```


OUTPUT

```
Invalid marks 150 for Kabir (ignored)
Invalid marks -5 for Kabir (ignored)
Final marks: 87
```

Watch out

Do not write a getter **and** a plain setter for every member automatically. A setter that just does `balance = b;` with no checks gives the same freedom as making the member public, and encapsulation is lost.

Ask instead: *what operations make sense?* A bank account needs `deposit()` and `withdraw()`, not `setBalance()`.

3.4 Why encapsulation matters

Benefit	Explanation
Data protection	Invalid values (negative balance, marks of 150) can be rejected in one place.
Easy debugging	If the balance is wrong, the bug must be inside <code>BankAccount</code> , because nothing else can change it.
Freedom to change internals	You could store the balance in paise as an <code>int</code> instead of a <code>double</code> . As long as the public methods behave the same, no other code needs to change.
Simpler for users	Users of the class only learn a few public methods, not the internal details.
Read-only or write-only data	Provide only a getter to make a value read-only from outside.

3.5 Encapsulation vs data hiding

These terms are often used together. **Data hiding** is the act of making members `private`. **Encapsulation** is the bigger idea: bundling data with its methods *and* controlling access to it. Data hiding is one of the tools used to achieve encapsulation.

Pillar 1 recap

- Encapsulation = bundle data + methods, and restrict direct access to the data.
- Make data members `private`; expose meaningful `public` methods.
- Validate inside setters and other methods so objects are always valid.
- Result: safer data, easier debugging, freedom to change internals later.

4. Inheritance

Inheritance lets a new class (the **derived** or **child** class) acquire the data and methods of an existing class (the **base** or **parent** class). The child can then add its own members or change inherited behaviour. It models an **is-a** relationship: a Dog *is an* Animal, a Car *is a* Vehicle, a SavingsAccount *is a* BankAccount.

Real-life analogy

Children inherit traits from their parents (eye colour, height) and then develop their own skills too.

A smartphone **is a** phone: it can still make calls (inherited), but adds a camera, apps and internet (its own features). Engineers did not reinvent calling from scratch.

Why this matters

Without inheritance, **Dog**, **Cat** and **Bird** would each repeat the same **name**, **eat()** and **sleep()** code. Fix a bug in **eat()** and you must remember to fix it in three places. With inheritance, common code is written once in **Animal** and every child gets it automatically.

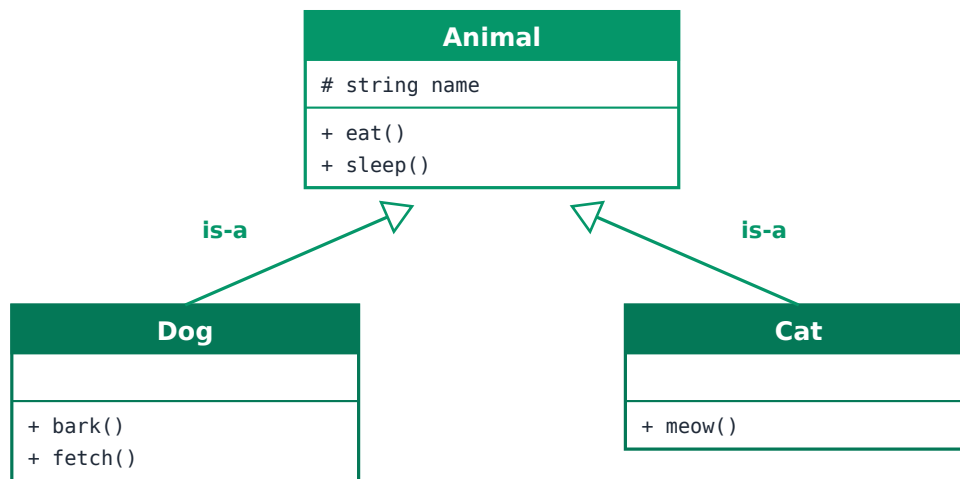
4.1 Syntax

C++ · syntax

```
class Child : public Parent {
    // extra members for the child
};
```

The **: public Parent** part says "Child inherits from Parent". Inside the child's constructor you call the parent's constructor in the initializer list, like **Dog(string n) : Animal(n) {}**, so the parent part of the object gets set up properly.

4.2 Example: animals



= protected + = public hollow arrow points to the parent

Figure 4.1 Dog and Cat inherit name, eat() and sleep() from Animal, and add their own methods.

C++ · animal.cpp

```

#include <iostream>
#include <string>
using namespace std;

// Base class (parent): what ALL animals have in common
class Animal {
protected:           // visible to child classes, hidden from outside
    string name;

public:
    Animal(string n) : name(n) {}

    void eat() { cout << name << " is eating." << endl; }
    void sleep() { cout << name << " is sleeping." << endl; }
};

// Derived class (child): a Dog IS-AN Animal, plus extra things
class Dog : public Animal {
public:
    Dog(string n) : Animal(n) {}    // pass the name up to Animal

    void bark() { cout << name << " says: Woof!" << endl; }
    void fetch() { cout << name << " fetches the ball." << endl; }
};

class Cat : public Animal {
public:
    Cat(string n) : Animal(n) {}

    void meow() { cout << name << " says: Meow!" << endl; }
};

int main() {
    Dog d("Bruno");
    Cat c("Kitty");

    d.eat();           // inherited from Animal
    d.bark();          // Dog's own method
    d.fetch();

    c.sleep();         // inherited from Animal
  
```

C++ · animal.cpp

```
c.meow();    // Cat's own method
return 0;
}
```

OUTPUT

```
Bruno is eating.
Bruno says: Woof!
Bruno fetches the ball.
Kitty is sleeping.
Kitty says: Meow!
```

- `Dog` never defined `eat()`, yet `d.eat()` works, because it was inherited from `Animal`.
- `name` is `protected` in `Animal`, so `Dog::bark()` can use it, but `main()` cannot.
- A `Dog` can `bark()`; a `Cat` cannot. Children add their own behaviour on top of what they inherit.

4.3 Modes of inheritance

The word before the parent's name (`public`, `protected` or `private`) controls how inherited members appear in the child. In practice you will use `public` inheritance over 95% of the time, because it is the one that means is-a.

Member in parent	public inheritance	protected inheritance	private inheritance
public	stays public	becomes protected	becomes private
protected	stays protected	stays protected	becomes private
private	not accessible	not accessible	not accessible

Note

Private members of the parent **are** inherited (they exist inside the child object and take memory), but the child cannot access them by name. It must use the parent's public or protected methods.

4.4 Types of inheritance

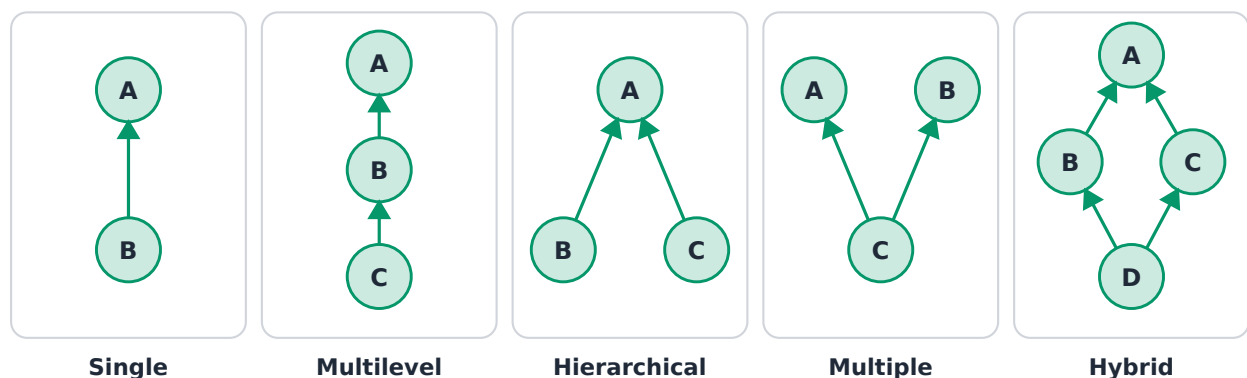


Figure 4.2 The five types of inheritance. Arrows point from child to parent.

Type	Shape	Example
Single	One parent, one child	Dog : Animal
Multilevel	A chain: grandparent, parent, child	ElectricCar : Car : Vehicle
Hierarchical	One parent, many children	Dog, Cat, Bird all from Animal
Multiple	One child, several parents (C++ allows this; Java does not for classes)	AllInOneMachine : Printer, Scanner
Hybrid	A mix of the above, often a diamond	TeachingAssistant : Student, Teacher, both from Person

4.5 Constructor and destructor order

When you create a child object, the parent part is built **first**, from the top of the hierarchy down. Destruction happens in the **exact reverse** order. This makes sense: a child may depend on its parent's data, so the parent must exist first and be destroyed last.

C++ · order.cpp

```
#include <iostream>
using namespace std;

class Vehicle {
public:
    Vehicle() { cout << "1. Vehicle constructor" << endl; }
    ~Vehicle() { cout << "    Vehicle destructor" << endl; }
};

class Car : public Vehicle { // Car IS-A Vehicle
public:
    Car() { cout << "2. Car constructor" << endl; }
    ~Car() { cout << "    Car destructor" << endl; }
};

class ElectricCar : public Car { // ElectricCar IS-A Car
public:
    ElectricCar() { cout << "3. ElectricCar constructor" << endl; }
    ~ElectricCar() { cout << "    ElectricCar destructor" << endl; }
};

int main() {
    cout << "Creating an ElectricCar:" << endl;
    {
        ElectricCar tesla;
        cout << "Object is ready. Leaving scope..." << endl;
    }
    return 0;
}
```

OUTPUT

```
Creating an ElectricCar:
1. Vehicle constructor
2. Car constructor
3. ElectricCar constructor
Object is ready. Leaving scope...
    ElectricCar destructor
    Car destructor
    Vehicle destructor
```

4.6 Multiple inheritance

C++ allows one class to inherit from several parents. An all-in-one machine is both a printer and a scanner:

C++ · multiple.cpp

```
#include <iostream>
using namespace std;

class Printer {
public:
    void print() { cout << "Printing document..." << endl; }
};

class Scanner {
public:
    void scan() { cout << "Scanning document..." << endl; }
};

// Multiple inheritance: one child, two parents
class AllInOneMachine : public Printer, public Scanner {
public:
    void photocopy() {
        cout << "Photocopy = ";
        scan();
        cout << "          then ";
        print();
    }
};

int main() {
    AllInOneMachine m;
    m.print();
    m.scan();
    m.photocopy();
    return 0;
}
```

OUTPUT

```
Printing document...
Scanning document...
Photocopy = Scanning document...
            then Printing document...
```

The diamond problem

Suppose `Student` and `Teacher` both inherit from `Person`, and `TeachingAssistant` inherits from both. Normally the TA would contain **two** separate `Person` parts, so `ta.age` would be ambiguous and the code would not compile. C++ solves this with **virtual inheritance**: writing `virtual public Person` tells the compiler to share a single `Person` part.

C++ · diamond.cpp

```
#include <iostream>
using namespace std;

class Person {
public:
    int age = 0;
    Person() { cout << "Person constructor" << endl; }
};

// 'virtual' inheritance: share ONE Person inside TeachingAssistant
class Student : virtual public Person {};
class Teacher : virtual public Person {};

class TeachingAssistant : public Student, public Teacher {};

int main() {
    TeachingAssistant ta;
    ta.age = 24;           // not ambiguous, because there is only one Person
    cout << "TA age: " << ta.age << endl;
    return 0;
}
```

OUTPUT

```
Person constructor
TA age: 24
```

Notice the `Person` constructor ran only once, proving there is a single shared `Person` inside the TA.

4.7 Inheritance vs composition (is-a vs has-a)

Inheritance is powerful but easy to overuse. Ask: is the relationship really **is-a**? A car is not an engine; a car **has an** engine. For has-a relationships, use **composition**: put an object of one class inside another as a member.

C++ · composition.cpp

```
#include <iostream>
#include <string>
using namespace std;

class Engine {
public:
    int horsepower;
    Engine(int hp) : horsepower(hp) {}
    void start() {
        cout << "Engine (" << horsepower << " hp) started" << endl;
    }
};

class Car {
private:
```

C++ · composition.cpp

```
string model;
Engine engine;          // Car HAS-AN Engine (composition)
public:
    Car(string m, int hp) : model(m), engine(hp) {}
    void drive() {
        engine.start();  // the car uses its engine
        cout << model << " is moving" << endl;
    }
};

int main() {
    Car c("Nexon", 120);
    c.drive();
    return 0;
}
```

OUTPUT

```
Engine (120 hp) started
Nexon is moving
```

	Inheritance	Composition
Relationship	is-a (Dog is an Animal)	has-a (Car has an Engine)
How	<code>class Dog : public Animal</code>	<code>Engine engine;</code> as a member
Coupling	Tight: child depends on parent's internals	Loose: uses only the member's public methods
Use when	Child truly is a specialised kind of parent	One thing is built from or uses another

Tip

A widely followed design guideline is **prefer composition over inheritance**. Reach for inheritance when the is-a test passes and you want polymorphism (next chapter).

4.8 What is not inherited

- Constructors and destructors (the child calls them, but does not own them).
- The assignment operator `operator=`.
- Friend relationships (a friend of the parent is not automatically a friend of the child).
- Access to private members (they are present but not accessible by name).

Pillar 2 recap

- Inheritance creates an is-a relationship: `class Child : public Parent`.
- The child reuses the parent's code and can add or change behaviour.
- `protected` members are visible to children but hidden from outside code.
- Parents are constructed first and destroyed last.
- C++ supports multiple inheritance; use `virtual` inheritance to fix the diamond.
- For has-a relationships, use composition instead.

5. Polymorphism

Polymorphism comes from Greek: *poly* = many, *morph* = form. It means **one name or interface can take many forms**. The same function call or operator behaves differently depending on the type of data or object it is used with.

Real-life analogy

One person plays many roles: a teacher at school, a parent at home, a customer at a shop. Same person, different behaviour depending on the situation.

Press the **Power** button on a TV, a laptop and a fan. Same action, but each device responds in its own way.

Even C++ itself is polymorphic: `+` adds numbers (`2 + 3`) but joins strings (`"Hi" + name`).

5.1 Two kinds of polymorphism

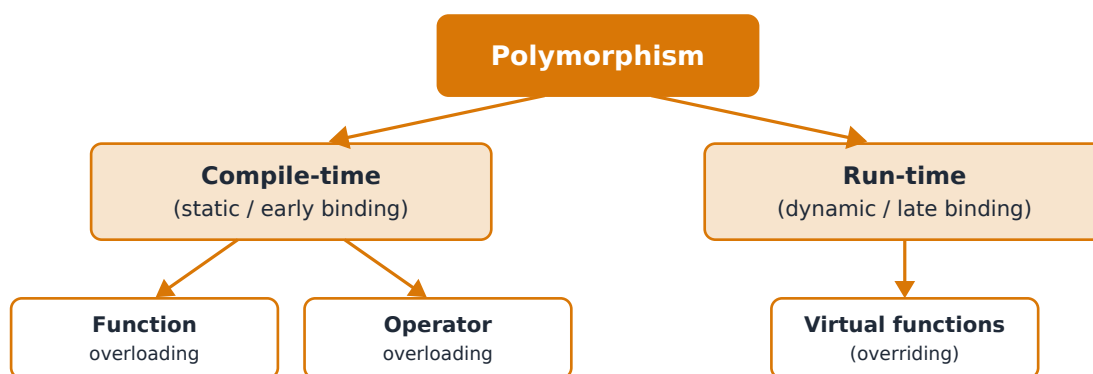


Figure 5.1 Compile-time vs run-time polymorphism.

	Compile-time	Run-time
Also called	Static polymorphism, early binding	Dynamic polymorphism, late binding
Decided	By the compiler, before the program runs	While the program is running
Achieved by	Function overloading, operator overloading (and templates)	Virtual functions overridden in derived classes
Needs inheritance?	No	Yes
Speed	Slightly faster (no lookup)	Tiny cost: one extra lookup per call

5.2 Compile-time: function overloading

Overloading means writing several functions with the **same name** but **different parameter lists** (different number or types of parameters). The compiler picks the right version by looking at the arguments you pass.

C++ · overload.cpp

```
#include <iostream>
#include <string>
using namespace std;

class Calculator {
public:
    // Same name 'add', different parameter lists
    int add(int a, int b)          { return a + b; }
    int add(int a, int b, int c)   { return a + b + c; }
    double add(double a, double b) { return a + b; }
    string add(string a, string b) { return a + b; }
};

int main() {
    Calculator calc;
    cout << calc.add(2, 3)          << endl; // calls add(int,int)
    cout << calc.add(2, 3, 4)       << endl; // calls add(int,int,int)
    cout << calc.add(2.5, 1.25)     << endl; // calls add(double,double)
    cout << calc.add(string("Hello, "), string("World")) << endl;
    return 0;
}
```

OUTPUT

```
5
9
3.75
Hello, World
```

Watch out

Functions cannot be overloaded by return type alone. `int get()` and `double get()` with identical parameters is an error, because a call like `get();` gives the compiler no way to choose.

5.3 Compile-time: operator overloading

C++ lets you define what operators like `+`, `==` or `<<` mean for your own classes. This makes code using your class read naturally:

C++ · operator.cpp

```
#include <iostream>
using namespace std;

class Money {
private:
    int rupees, paise;
public:
    Money(int r, int p) : rupees(r), paise(p) {}

    // Teach '+' what it means to add two Money objects
    Money operator+(const Money& other) const {
        int totalPaise = (rupees + other.rupees) * 100
            + paise + other.paise;
        return Money(totalPaise / 100, totalPaise % 100);
    }

    // Teach '==' how to compare two Money objects
    bool operator==(const Money& other) const {
        return rupees == other.rupees && paise == other.paise;
    }
}
```

C++ · operator.cpp

```
}

void print() const {
    cout << "Rs " << rupees << "." << (paise < 10 ? "0" : "")
        << paise << endl;
}

};

int main() {
    Money lunch(120, 75);
    Money snack(45, 50);
    Money total = lunch + snack;    // uses our operator+
    total.print();

    cout << boolalpha << (total == Money(166, 25)) << endl;
    return 0;
}
```

OUTPUT

```
Rs 166.25
true
```

Without operator overloading you would have to write something like `Money total = lunch.add(snack);`. Both work, but `lunch + snack` is clearer.

5.4 Run-time: the problem without `virtual`

Run-time polymorphism is where OOP gets truly powerful. It relies on one fact: **a base-class pointer (or reference) can point to a derived-class object**, because a Circle *is* a Shape. But watch what happens if the function is not marked `virtual`:

C++ · novirtual.cpp

```
#include <iostream>
using namespace std;

class Shape {
public:
    void draw() { cout << "Drawing a generic shape" << endl; } // NOT virtual
};

class Circle : public Shape {
public:
    void draw() { cout << "Drawing a circle" << endl; }
};

int main() {
    Circle c;
    Shape* s = &c;    // base-class pointer to a Circle
    s->draw();        // which draw() runs?
    return 0;
}
```

OUTPUT

```
Drawing a generic shape
```

We pointed at a Circle, yet the Shape version ran. Without `virtual`, C++ uses **static binding**: it looks at the type of the *pointer* (`Shape*`) at compile time and calls `Shape::draw()`. We want

the decision based on the type of the *actual object*.

5.5 Run-time: virtual functions to the rescue

Mark the function `virtual` in the base class, and override it in the children. Now C++ decides at run time, based on the real object:

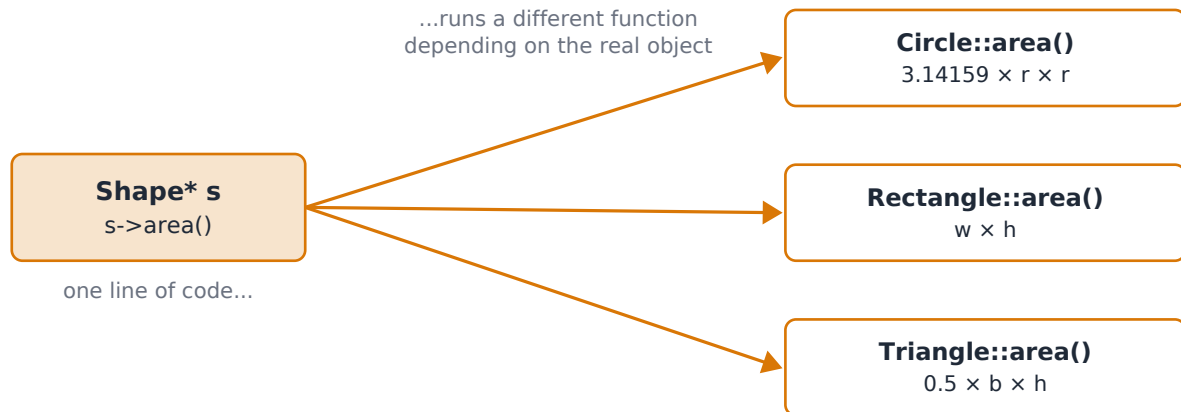


Figure 5.2 The same call, `s->area()`, runs different code for each kind of shape.

C++ · shapes.cpp

```
#include <iostream>
#include <iomanip>
#include <string>
#include <vector>
using namespace std;

class Shape {
public:
    virtual double area() const { return 0; }           // can be overridden
    virtual string name() const { return "Shape"; }
    virtual ~Shape() {}                               // always do this!
};

class Circle : public Shape {
    double r;
public:
    Circle(double radius) : r(radius) {}
    double area() const override { return 3.14159 * r * r; }
    string name() const override { return "Circle"; }
};

class Rectangle : public Shape {
    double w, h;
public:
    Rectangle(double width, double height) : w(width), h(height) {}
    double area() const override { return w * h; }
    string name() const override { return "Rectangle"; }
};

class Triangle : public Shape {
    double b, h;
public:
    Triangle(double base, double height) : b(base), h(height) {}
    double area() const override { return 0.5 * b * h; }
    string name() const override { return "Triangle"; }
};
```

C++ · shapes.cpp

```
int main() {
    vector<Shape*> shapes = {
        new Circle(2), new Rectangle(3, 4), new Triangle(3, 6)
    };

    cout << fixed << setprecision(2);
    for (Shape* s : shapes) {
        // ONE line of code, but the right area() is picked at run time
        cout << setw(10) << left << s->name()
            << " area = " << s->area() << endl;
    }

    for (Shape* s : shapes) delete s;
    return 0;
}
```

OUTPUT

```
Circle    area = 12.57
Rectangle area = 12.00
Triangle  area = 9.00
```

The loop has no **if**, no **switch** and no idea which shapes exist. Tomorrow you can add a **Hexagon** class and this loop works unchanged. This is the biggest practical benefit of polymorphism: **code that works with types that did not exist when it was written.**

Things to notice

- **virtual** in the base class enables run-time lookup for that function.
- **override** (C++11) in the child asks the compiler to check you really are overriding a base virtual function. A typo like **aera()** then becomes a compile error instead of a silent bug. Always use it.
- **virtual ~Shape() {}** is a **virtual destructor**. Without it, **delete s** on a **Shape*** that points to a **Circle** would only run **Shape's** destructor, which is undefined behaviour and can leak resources. **Rule: if a class has any virtual function, give it a virtual destructor.**

5.6 How does C++ do this? (vtable, simply)

For every class with virtual functions, the compiler builds a hidden table of function addresses called the **vtable**. Each object of that class carries a hidden pointer (the **vptr**) to its class's vtable. When you call **s->area()**, the program follows the object's vptr to its vtable and jumps to the **area()** stored there: **Circle's** table holds **Circle::area**, **Rectangle's** holds **Rectangle::area**. That one extra lookup is the small cost of run-time polymorphism.

5.7 Overloading vs overriding

These two words sound alike and are the most commonly confused pair in OOP interviews:

	Overloading	Overriding
What	Same name, different parameters	Same name, same parameters, in a child class
Where	Usually in the same class	Across parent and child classes

	Overloading	Overriding
Inheritance needed?	No	Yes
Keyword	None	<code>virtual</code> in parent, <code>override</code> in child
Resolved	Compile time	Run time
Purpose	Convenience: one name for related operations	Let children give their own version of a behaviour
Example	<code>add(int,int)</code> and <code>add(double,double)</code>	<code>Circle::area()</code> replaces <code>Shape::area()</code>

Pillar 3 recap

- Polymorphism = one interface, many forms.
- Compile-time: function overloading and operator overloading.
- Run-time: `virtual` functions + base-class pointers/references + overriding.
- Without `virtual`, the pointer's type decides; with `virtual`, the object's type decides.
- Always use `override` in children and a `virtual` destructor in the base.

6. Abstraction

Abstraction means showing only the essential features of something and hiding the complex details of how it works. It answers *what* an object does, not *how* it does it. Users of a class work with a simple, clear interface and never need to know what happens inside.

Real-life analogy

To drive a car you use a steering wheel, accelerator and brake. You do not need to understand fuel injection or gearbox design. The car abstracts those away.

A TV remote has a few buttons; the electronics behind them are hidden.

When you call `cout << x;` or `sort(v.begin(), v.end());` you are using abstraction: you know what they do, not the hundreds of lines inside.

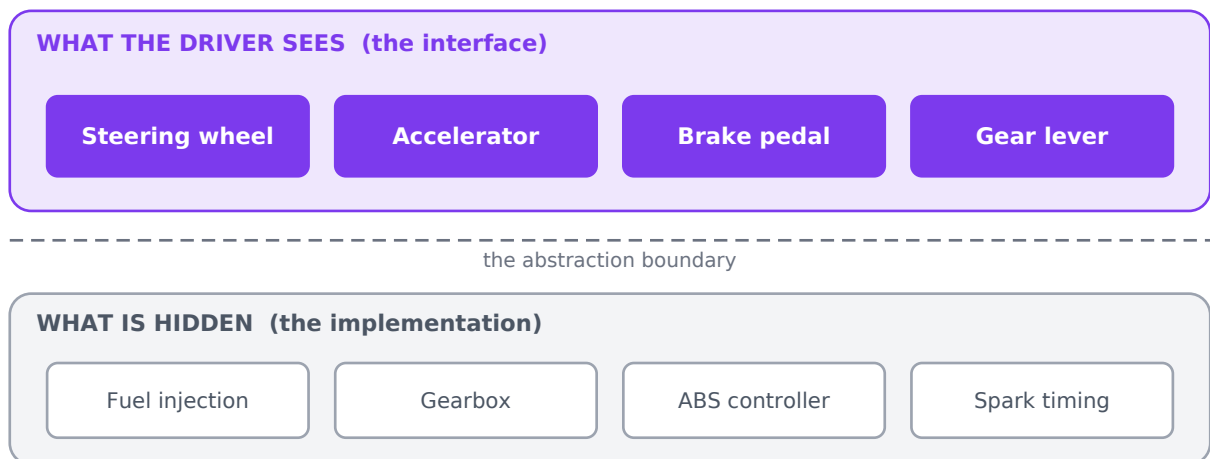


Figure 6.1 Abstraction separates what you use from how it works.

6.1 Abstract classes and pure virtual functions

In C++, abstraction is mainly expressed through **abstract classes**. A **pure virtual function** is a virtual function with `= 0` instead of a body. It declares *what* must exist without saying *how*:

C++ · syntax

```
virtual double calculateSalary() const = 0;    // pure virtual
```

A class with at least one pure virtual function is **abstract**: you cannot create objects of it directly. Each child class must provide a body for every pure virtual function, otherwise the child is abstract too.

C++ · employee.cpp

```
#include <iostream>
#include <string>
using namespace std;

// Abstract class: has at least one PURE virtual function (= 0).
```

C++ · employee.cpp

```
// You cannot create an Employee directly - only its children.
class Employee {
protected:
    string name;
public:
    Employee(string n) : name(n) {}
    virtual ~Employee() {}

    // WHAT every employee must provide - no HOW here
    virtual double calculateSalary() const = 0;

    // Shared, concrete behaviour that uses the abstract part
    void printPayslip() const {
        cout << name << " -> Rs " << calculateSalary() << endl;
    }
};

class FullTimeEmployee : public Employee {
    double monthlySalary;
public:
    FullTimeEmployee(string n, double s) : Employee(n), monthlySalary(s) {}
    double calculateSalary() const override { return monthlySalary; }
};

class PartTimeEmployee : public Employee {
    int hours;
    double ratePerHour;
public:
    PartTimeEmployee(string n, int h, double r)
        : Employee(n), hours(h), ratePerHour(r) {}
    double calculateSalary() const override { return hours * ratePerHour; }
};

int main() {
    // Employee e("X");    // COMPILE ERROR: Employee is abstract

    FullTimeEmployee a("Anjali", 60000);
    PartTimeEmployee b("Rohan", 80, 350);

    Employee* staff[] = { &a, &b };
    for (Employee* e : staff) {
        e->printPayslip();
    }
    return 0;
}
```

OUTPUT

```
Anjali -> Rs 60000
Rohan -> Rs 28000
```

- **Employee** says every employee has a salary but leaves the formula to each child. Full-timers get a fixed amount; part-timers get hours × rate.
- `printPayslip()` is written **once** in the base class and works for every kind of employee, because it calls the abstract `calculateSalary()`.
- `Employee e("X");` does not compile: cannot declare variable 'e' to be of abstract type 'Employee'. But pointers and references to **Employee** are fine, which is how we use it polymorphically.

6.2 Interfaces in C++

C++ has no **interface** keyword (Java and C# do). Instead, a class whose functions are **all** pure virtual, with no data members, acts as an interface: a pure contract listing what an object can do.

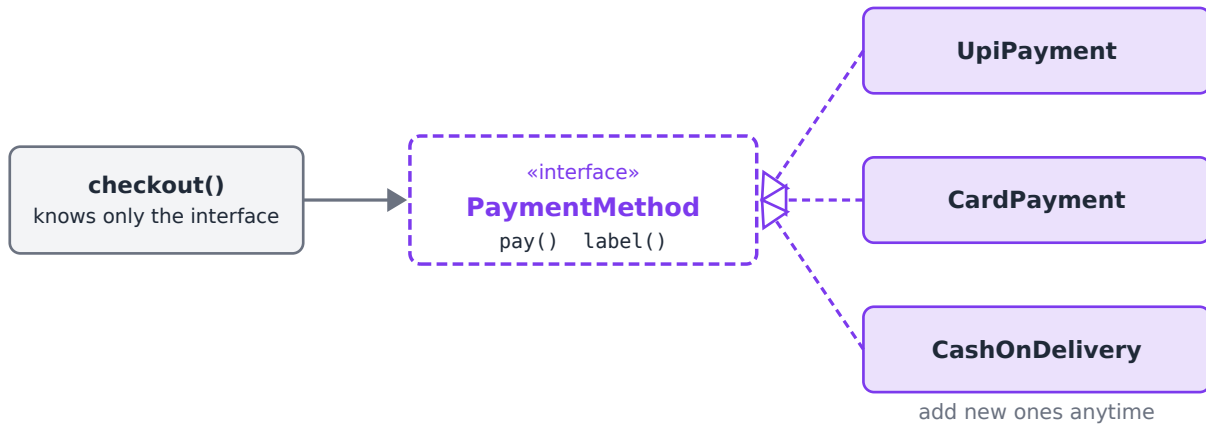


Figure 6.2 checkout() depends only on the PaymentMethod interface, never on a specific payment type.

C++ · payment.cpp

```
#include <iostream>
#include <string>
using namespace std;

// An "interface" in C++: a class with ONLY pure virtual functions.
class PaymentMethod {
public:
    virtual bool pay(double amount) = 0;
    virtual string label() const = 0;
    virtual ~PaymentMethod() {}
};

class UpiPayment : public PaymentMethod {
    string upiId;
public:
    UpiPayment(string id) : upiId(id) {}
    bool pay(double amount) override {
        cout << " UPI request of Rs " << amount << " to " << upiId << endl;
        return true;
    }
    string label() const override { return "UPI"; }
};

class CardPayment : public PaymentMethod {
    string last4;
public:
    CardPayment(string l4) : last4(l4) {}
    bool pay(double amount) override {
        cout << " Charging card ending " << last4 << endl;
        return amount <= 50000; // pretend card limit
    }
    string label() const override { return "Card"; }
};

class CashOnDelivery : public PaymentMethod {
public:
    bool pay(double amount) override {
        cout << " Collect Rs " << amount << " in cash at the door" << endl;
    }
};
```

C++ · payment.cpp

```
        return true;
    }
    string label() const override { return "Cash on Delivery"; }
};

// Checkout knows NOTHING about UPI, cards or cash.
// It only knows the PaymentMethod interface.
void checkout(PaymentMethod& method, double total) {
    cout << "Paying Rs " << total << " via " << method.label() << endl;
    if (method.pay(total)) cout << " Order placed!" << endl;
    else cout << " Payment failed." << endl;
}

int main() {
    UpiPayment upi("riya@okbank");
    CardPayment card("4242");
    CashOnDelivery cod;

    checkout(upi, 799);
    checkout(card, 65000);
    checkout(cod, 250);
    return 0;
}
```

OUTPUT

```
Paying Rs 799 via UPI
  UPI request of Rs 799 to riya@okbank
  Order placed!
Paying Rs 65000 via Card
  Charging card ending 4242
  Payment failed.
Paying Rs 250 via Cash on Delivery
  Collect Rs 250 in cash at the door
  Order placed!
```

This example shows why abstraction is so valuable in real software:

- `checkout()` was written once and **never needs to change**. Adding a `WalletPayment` class next month only means writing one new class.
- The team writing `checkout()` and the team writing `UpiPayment` can work in parallel; they only need to agree on the interface.
- Each payment type hides its messy details (bank APIs, card networks) behind `pay()`.
- This idea, **open for extension, closed for modification**, is one of the most important design principles in software (the "O" in SOLID).

6.3 Abstract class vs interface

	Abstract class	Interface (in C++)
Pure virtual functions	At least one	All functions
Normal methods with code	Allowed (<code>printPayslip()</code>)	None (by convention)
Data members	Allowed (<code>name</code>)	None (by convention)
Constructor	Can have one	Usually none needed
Use when	Children share common code and data	You only want to define a capability/contract
Example	<code>Employee</code>	<code>PaymentMethod</code>

6.4 Abstraction vs encapsulation

Beginners often mix these two up, because both involve "hiding". The difference is *what* is hidden and *why*:

	Encapsulation	Abstraction
Hides	Data (the internal state)	Complexity (the implementation details)
Main question	How do I protect my data?	What should the user see?
Level	Implementation level	Design level
C++ tools	<code>private</code> , <code>protected</code> , getters/setters	Abstract classes, pure virtual functions, interfaces, header files
Analogy	The car's engine is sealed so you cannot fiddle with it	You drive with a steering wheel without knowing how the engine works

They work together: abstraction decides what the public interface should be, and encapsulation enforces that everything else stays private.

Pillar 4 recap

- Abstraction = show what an object does, hide how it does it.
- Pure virtual function: `virtual void f() = 0;`
- A class with any pure virtual function is abstract and cannot be instantiated.
- A class with only pure virtual functions acts as an interface.
- Code written against an abstraction keeps working when new types are added.

7. Putting It All Together: A Library System

Real programs use all four pillars at once. This small library system issues books and DVDs and calculates late fees. Read the code first, then use the table below to spot each pillar.

Pillar	Where it appears in the code
Encapsulation	<code>id</code> , <code>title</code> and <code>issued</code> are private. The only way to change <code>issued</code> is <code>issue()</code> , which refuses to issue an item twice.
Abstraction	<code>LibraryItem</code> is abstract. It declares <code>loanDays()</code> and <code>type()</code> as pure virtual, so every kind of item must define them. <code>Library</code> only ever works with <code>LibraryItem</code> .
Inheritance	<code>Book</code> and <code>Dvd</code> inherit <code>id</code> , <code>title</code> , issuing logic and the default late fee from <code>LibraryItem</code> instead of rewriting it.
Polymorphism	<code>item->lateFee()</code> and <code>item->loanDays()</code> run the <code>Book</code> or <code>DVD</code> version automatically. <code>DVDs</code> override <code>lateFee()</code> to charge more.

C++ · library.cpp

```
#include <iostream>
#include <iomanip>
#include <memory>
#include <string>
#include <vector>
using namespace std;

// ----- ABSTRACTION + ENCAPSULATION -----
class LibraryItem {
private:
    string id, title;
    bool issued = false; // hidden state
public:
    LibraryItem(string i, string t) : id(i), title(t) {}
    virtual ~LibraryItem() {}

    string getId() const { return id; }
    string getTitle() const { return title; }

    bool issue() { // controlled change
        if (issued) return false;
        issued = true;
        return true;
    }
    void giveBack() { issued = false; }

    virtual int loanDays() const = 0; // every item MUST define
    virtual string type() const = 0;
    virtual double lateFee(int daysLate) const { // items MAY change
        return daysLate * 2.0;
    }
};

// ----- INHERITANCE -----
class Book : public LibraryItem {
    string author;
```

C++ · library.cpp

```
public:
    Book(string i, string t, string a) : LibraryItem(i, t), author(a) {}
    int loanDays() const override { return 14; }
    string type() const override { return "Book"; }
};

class Dvd : public LibraryItem {
public:
    Dvd(string i, string t) : LibraryItem(i, t) {}
    int loanDays() const override { return 3; }
    string type() const override { return "DVD"; }
    double lateFee(int daysLate) const override { // POLYMORPHISM
        return daysLate * 10.0; // DVDs cost more
    }
};

class Library {
    vector<unique_ptr<LibraryItem>> items;
public:
    void add(LibraryItem* item) { items.emplace_back(item); }

    void issue(const string& id) {
        for (auto& item : items) {
            if (item->getId() == id) {
                if (item->issue())
                    cout << "Issued " << item->getTitle() << " for "
                        << item->loanDays() << " days" << endl;
                else
                    cout << item->getTitle() << " is already issued"
                        << endl;
                return;
            }
        }
        cout << "No item with id " << id << endl;
    }

    void lateFeeReport(int daysLate) const {
        cout << "\nLate fee if returned " << daysLate
            << " days late:" << endl;
        cout << fixed << setprecision(2);
        for (const auto& item : items) {
            cout << " " << setw(5) << left << item->type()
                << setw(18) << left << item->getTitle()
                << "Rs " << item->lateFee(daysLate) << endl;
        }
    }
};

int main() {
    Library lib;
    lib.add(new Book("B1", "Clean Code", "Robert Martin"));
    lib.add(new Book("B2", "Let Us C++", "Yashavant Kanetkar"));
    lib.add(new Dvd("D1", "Planet Earth"));

    lib.issue("B1");
    lib.issue("D1");
    lib.issue("B1"); // already issued
    lib.issue("X9"); // does not exist
    lib.lateFeeReport(5);
    return 0;
}
```

OUTPUT

```
Issued Clean Code for 14 days
Issued Planet Earth for 3 days
Clean Code is already issued
No item with id X9
```

```
Late fee if returned 5 days late:
Book Clean Code      Rs 10.00
Book Let Us C++      Rs 10.00
DVD Planet Earth     Rs 50.00
```

Walkthrough

1. `Library` stores items as `unique_ptr<LibraryItem>`, smart pointers that delete the objects automatically when the library is destroyed. That is why there is no `delete` anywhere.
2. `issue("B1")` finds the Book and calls `item->issue()`. The private `issued` flag becomes true. Issuing B1 again is refused, protected by encapsulation.
3. `item->loanDays()` prints 14 for the book and 3 for the DVD: run-time polymorphism through the abstract function.
4. In the fee report, both books use the inherited `lateFee()` ($5 \times 2 = 10$) while the DVD uses its overridden version ($5 \times 10 = 50$).
5. To add magazines, you would write one new `Magazine` class. `Library` would not change at all.

Tip

Try it: add a `Magazine` class with a 7-day loan and a late fee of Rs 1 per day, then add one to the library in `main()`. You will not need to touch `Library` or `LibraryItem`.

8. The Four Pillars Side by Side

Use this chapter for quick revision before an exam or interview.

	Encapsulation	Inheritance	Polymorphism	Abstraction
One line	Protect data inside a class	Reuse and extend a class	One interface, many forms	Show what, hide how
Key question	Who may change this data?	Is X a kind of Y?	Which version should run?	What does the user need to see?
C++ tools	<code>private</code> , <code>public</code> , getters, setters	<code>class B : public A</code> , <code>protected</code>	overloading, <code>virtual</code> , <code>override</code>	pure virtual = 0, abstract classes
Analogy	ATM / capsule	Parent and child	Power button on many devices	Driving a car
Main benefit	Safety, easy debugging	Less duplicate code	Flexible, extensible code	Simplicity, loose coupling
Example in this guide	<code>BankAccount</code>	<code>Dog : Animal</code>	<code>Shape::area()</code>	<code>Employee</code> , <code>PaymentMethod</code>

8.1 How they work together

The pillars are not separate tricks; they support each other:

- **Encapsulation** makes each class a safe, self-contained unit.
- **Inheritance** connects those units into families that share code.
- **Polymorphism** lets code treat every member of a family the same way while each behaves correctly.
- **Abstraction** defines the clean contract at the top of the family, so the rest of the program depends on the contract rather than on details.

Run-time polymorphism in C++ needs inheritance (a base class) and is most useful with abstraction (a base class that defines a contract). That is why the Shape, Employee, Payment and Library examples all use several pillars at once.

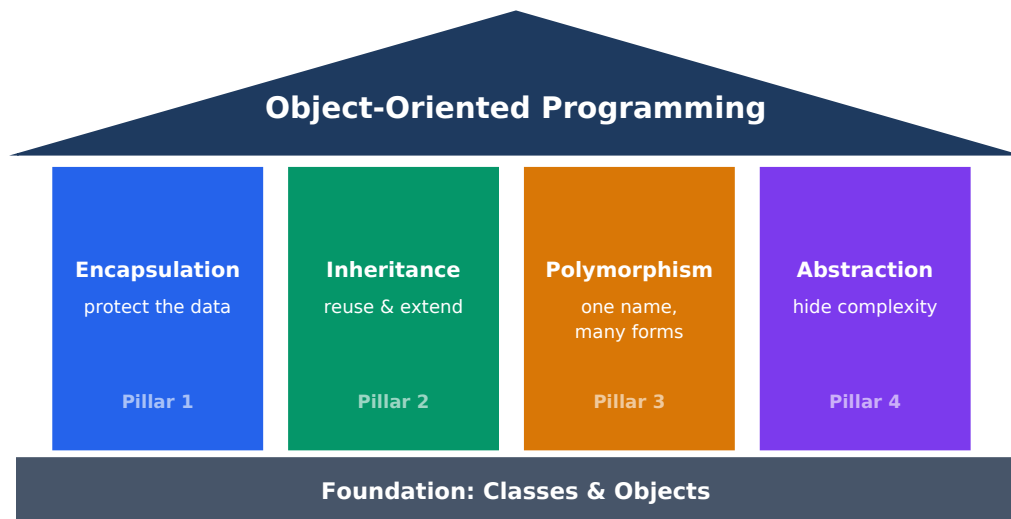


Figure 8.1 The four pillars stand on classes and objects and together hold up OOP.

9. Interview Questions and Answers

Q1. What is OOP?

A programming paradigm that organises software around objects, which bundle data (attributes) and behaviour (methods). It rests on four pillars: encapsulation, inheritance, polymorphism and abstraction.

Q2. What is the difference between a class and an object?

A class is a blueprint that uses no memory for data by itself. An object is an instance of a class that occupies memory and holds actual values. One class can create many objects.

Q3. Name and define the four pillars.

Encapsulation: hide data and expose controlled methods. Inheritance: a class acquires members of another (is-a). Polymorphism: one interface, many forms. Abstraction: expose what, hide how.

Q4. What is the difference between encapsulation and abstraction?

Encapsulation hides data to protect it (implementation level: `private`, getters/setters). Abstraction hides complexity to simplify use (design level: abstract classes, interfaces).

Q5. What are access specifiers? What is the default in a class?

`public`, `protected` and `private` control who can access members. The default is `private` in a `class` and `public` in a `struct`.

Q6. What is a constructor? Can it be overloaded? Can it be virtual?

A special method with the class's name and no return type that initialises an object. It can be overloaded (default, parameterized, copy). It cannot be virtual.

Q7. Why should a base class have a virtual destructor?

So that deleting a derived object through a base pointer calls the derived destructor first. Without it, only the base destructor runs, which is undefined behaviour and can leak resources.

Q8. What is the difference between overloading and overriding?

Overloading: same name, different parameters, resolved at compile time, no inheritance needed. Overriding: a child redefines a base `virtual` function with the same signature, resolved at run time.

Q9. What is a virtual function?

A member function declared with `virtual` in the base class whose call is resolved at run time based on the actual object type, enabling run-time polymorphism.

Q10. What is a pure virtual function and an abstract class?

A pure virtual function has `= 0` and no body. A class with at least one is abstract: it cannot be instantiated, but you can have pointers and references to it.

Q11. What is the vtable?

A per-class table of pointers to virtual functions. Each object holds a hidden `vp`tr to its class's vtable, which is used to find the right function at run time.

Q12. What is early binding vs late binding?

Early (static) binding chooses the function at compile time (normal and overloaded functions). Late (dynamic) binding chooses at run time (virtual functions through pointers/references).

Q13. What is the diamond problem and how does C++ solve it?

When a class inherits from two classes that share a common base, it gets two copies of that base, making access ambiguous. C++ solves it with virtual inheritance (`class B : virtual public A`) so only one copy exists.

Q14. Does C++ support multiple inheritance?

Yes. Java and C# do not allow it for classes (they use interfaces instead).

Q15. What is the `this` pointer?

A pointer, available inside non-static member functions, to the object on which the function was called.

Q16. Composition vs inheritance: which should you prefer?

Inheritance models is-a; composition models has-a. Composition is generally preferred because it creates looser coupling; use inheritance when the is-a relationship is real and you need polymorphism.

Q17. What is a friend function? Does it break encapsulation?

A non-member function granted access to a class's private members with the `friend` keyword. It weakens encapsulation slightly, so use it sparingly, typically for operators like `<<`.

Q18. Can we create an object of an abstract class?

No. We can only create objects of concrete derived classes that implement all pure virtual functions.

10. Practice Exercises

Solve these in order. Each one practises a specific idea. Try first, and use the hint only if you are stuck for more than 15 minutes.

#	Exercise	Task	Hint
1	Rectangle Easy · Classes & objects	Create a Rectangle class with length and width , a constructor, and methods area() and perimeter() . Create three rectangles and print their areas.	Copy the structure of the Car example.
2	Student grades Easy · Encapsulation	Make a Student with private name and marks . setMarks() must reject values outside 0 to 100. Add getGrade() returning 'A' (90+), 'B' (75+), 'C' (50+) or 'F'.	Start from the setter example and add an if/else if chain.
3	Temperature Easy · Encapsulation	Store temperature privately in Celsius only. Provide getCelsius() , getFahrenheit() and setCelsius() . Reject values below -273.15.	$F = C \times 9 / 5 + 32$. Fahrenheit is calculated, never stored.
4	Vehicles Medium · Inheritance	Create Vehicle (brand, year, displayInfo()). Derive Car (doors) and Bike (hasCarrier). Each child adds its own display method. Print constructor messages to observe the order.	Call the parent constructor in the initializer list: Car(...) : Vehicle(b, y) .
5	More shapes Medium · Polymorphism	Add a Square and add a virtual perimeter() to the Shape example. Store several shapes in a vector<Shape*> and print the total area.	Remember override and the virtual destructor.
6	Fraction Medium · Operator overloading	Write a Fraction class (numerator, denominator) with operator+ , operator* and operator== . Always store fractions in lowest terms.	Use std::gcd from <numeric> (C++17) to reduce.
7	Wallet payment Medium · Abstraction	Add a WalletPayment class to the payment example that fails if the wallet balance is too low. Do not change checkout() at all.	Store the wallet balance as a private member; implement pay() and label() .
8	Interns Medium · Abstraction + polymorphism	Add an Intern employee with a fixed stipend and a Manager who gets a salary plus a bonus. Print paylips for a mixed array of employees.	Manager can inherit from FullTimeEmployee and add the bonus.
9	School management Challenge · All four pillars	Design a small system with an abstract Person (name, age, pure virtual role()), Student and Teacher children, and a School class that stores people and prints a report. Use encapsulation for all data.	Model it on the library project in Chapter 7.

Tip

After finishing an exercise, ask yourself: *Which pillar did I use, and what would break if I removed it?* Being able to answer that is the sign you have truly understood OOP.

11. Cheat Sheet and Glossary

11.1 Syntax at a glance

C++ · cheatsheet.cpp

```

class Base {                                // a class
private:                                   // only this class
    int secret;
protected:                               // this class + children
    string name;
public:                                    // everyone
    Base(string n) : name(n), secret(0) {} // constructor + init list
    virtual ~Base() {}                     // virtual destructor
    int getSecret() const { return secret; } // getter (const = read-only)
    void setSecret(int s) { if (s >= 0) secret = s; } // validated setter

    virtual void speak() { }               // virtual: can be overridden
    virtual double area() const = 0;        // pure virtual -> Base is abstract
};

class Derived : public Base {               // inheritance (is-a)
public:
    Derived(string n) : Base(n) {}          // call parent constructor
    void speak() override { }              // overriding (run-time polymorphism)
    double area() const override { return 1.0; }
    void show(int x) { }                    // overloading (compile-time):
    void show(double x) { }                 // same name, different parameters
};

Base* p = new Derived("x");                 // base pointer to derived object
p->speak();                                  // calls Derived::speak()
delete p;                                    // safe thanks to virtual destructor

```

11.2 Glossary

Term	Meaning
Abstract class	A class with at least one pure virtual function; cannot be instantiated.
Abstraction	Exposing what an object does while hiding how it does it.
Access specifier	<code>public</code> , <code>protected</code> or <code>private</code> ; controls who can use a member.
Attribute / data member	A variable that belongs to an object.
Base / parent class	The class being inherited from.
Binding	Connecting a function call to the function body: early (compile time) or late (run time).
Class	A blueprint that defines data and behaviour.
Composition	Building a class from objects of other classes (has-a).
Constructor	Special method that initialises a new object; same name as the class.
Derived / child class	A class that inherits from another.
Destructor	Special method <code>~ClassName()</code> that runs when an object is destroyed.
Encapsulation	Bundling data with methods and restricting direct access to the data.
Getter / setter	Methods to read / validate-and-write a private member.
Inheritance	Mechanism by which a class acquires another class's members (is-a).
Instance	Another word for an object of a class.
Interface	In C++, a class with only pure virtual functions; a contract.
Method / member function	A function that belongs to a class.
Object	A concrete instance of a class with its own state.
Overloading	Same function name, different parameter lists.
Overriding	A child class redefining a base class's virtual function.
Polymorphism	One interface, many forms of behaviour.
Pure virtual function	<code>virtual void f() = 0;</code> a function with no body that children must implement.
<code>this</code>	Pointer to the object on which a member function was called.
Virtual function	A function whose call is resolved at run time by the object's actual type.
vtable	Hidden per-class table of virtual function addresses used for late binding.

Where to go next

You now know what OOP is, why it exists, and how each of the four pillars works in C++. Good next topics: templates (generic programming), the STL containers (`vector`, `map`), smart pointers (`unique_ptr`, `shared_ptr`), exception handling, and the SOLID design principles.